Practical 4 – Problem sheet

This exercise sheet discusses one- and two-dimensional arrays. Hence, they involve functions that take arrays as input parameters. To enter an array in the BlueJ interface (when calling a function), type it as in the following example:

```
{3, 8 , -6, 15, 42}
```

Where functions take arrays as input parameters, let us adopt the convention (for this exercise sheet) that the function should *not* alter the values in the input arrays.

Exercise 1 (S)

In this exercise, you will practice working with arrays. The function definitions (with the correct signature, i.e., parameter and return types) and some template code are already given in the class `Arrays`. See the class `ArrayDemo` from Lesson 14 for examples.

Write functions that perform the following tasks:

(a) `sumOfThree(...)`: Given an array of three `int` numbers a_0, a_1, a_2, returns the expression $a_0 + 2a_1 + 3a_2$.

(b) `firstPlusLast(...)`: Given an array of `int` numbers, returns the sum of the first and the last element of the array.

(c) `dayNames()`: Returns an array of seven `String` values, containing the names of the day of the week (starting with Monday). Takes no parameters.

(d) `max(...)`: Given an array of `int` numbers, returns the largest element in the array. *Hint:* This involves a loop. Have a look at the class `ZetaExample` from Lesson 12 for an example on how to compute maxima.

(e) `min(...)`: Given an array of `int` numbers, returns the smallest element in the array. *Hint:* Very similar to the previous part.

(f) `reverse(...)`: Given an array of `int` numbers, returns the array in opposite order; i.e., the first element of the output array is the last element of the input array, etc.

In all cases where an array parameter is given, you may assume that the array is not `null` and has at least one element.

Exercise 2 (I)

For a data series $x = (x_0, \ldots, x_{n-1})$ of n real values, we define the following statistical quantities:

$$\text{the mean value,} \qquad \bar{x} := \frac{1}{n} \sum_{0 \le j < n} x_j, \tag{1}$$

$$\text{the } k\text{th moment (for } k \in \mathbb{N}\text{),} \qquad M_{k,x} := \frac{1}{n} \sum_{0 \le j < n} x_j^k, \tag{2}$$

$$\text{the standard deviation,} \qquad \sigma_x := \sqrt{\frac{1}{n-1} \sum_{0 \le j < n} (x_j - \bar{x})^2}. \tag{3}$$

In Java, let us represent these data series as an array of double-precision floating point numbers, i.e., as a variable of type `double[]`.

Your task is to write functions as follows:

(a) a function `meanValue` that computes the mean value of a data series. It should take one input parameter of type `double[]`, representing the data series. The return value should be of type `double`. *Hint:* See the code from Lesson 14, function `ArrayDemo.sumOfElements`, for the solution of a similar problem;

(b) a function `moment` that computes the kth moment of a data series. It should take two input parameters, one of type `double[]`, representing the data series, and one of type `int`, representing $k \geq 1$. The return value should be of type `double`;

(c) a function `standardDeviation` that computes the standard deviation of a data series. It should take one input parameter of type `double[]`, representing the data series. The return value should be of type `double`.

If, in the input parameters, the data series is `null` or if $n = 0$, then the functions should return `Double.NaN` as their result. In part c) the same applies if the length of the data series is $n = 1$.

Exercise 3 (I)

In Lesson 15, we developed some basic techniques to compute with vectors $x \in \mathbb{R}^n$. In this exercise, we are going to expand this to include more operations on vectors. As a reminder, given two vectors $x, y \in \mathbb{R}^n$ and a number $c \in \mathbb{R}$, the *scalar multiplication* of c and x, denoted $cx \in \mathbb{R}^n$, is defined as

$$(cx)_j = c\,x_j \quad (j = 0, \ldots, n-1), \tag{4}$$

and the *scalar product* of x and y is given by

$$x \cdot y = \sum_{0 \leq j < n} x_j y_j \ \in \mathbb{R}. \tag{5}$$

(As before, we label the vector components as $x_0, \ldots, x_{n-1}$ etc.)

The code used in the video lessons for generating a unit vector and for adding two vectors is provided in the class `Vectors`. Your task is to add the following functions to the class.

a) Write a function `scalarMult` that computes the scalar multiplication cx, given a number c and a vector x. Parameter types are: `double` (for c) and `double[]` (for x); return type: `double[]`. If the vector x is passed as `null`, then the return value should be `null`; if the array for x has length 0, then also the return value should be of length 0.

b) Write a function `scalarProduct` that computes the scalar product $x \cdot y$, given two vectors x and y. Parameter types are: `double[], double[]` (for the two vectors); return type: `double`. If any of the vectors x and y is passed as `null`, or if the two arrays do not have equal length, the return value should be `Double.NaN`. If x and y have length 0, the return value should be 0.

c) Let

$$a := (1, 2, 3, 4), \quad b := (0.4, 0.3, 0.2, 0.1), \quad c := (-1, 0, 3, 2), \quad m := 2.9.$$

Using the above, write a function `vectorComputation` that computes the expression $c \cdot (a + mb)$, and returns the result as a `double`. (The function should have no input parameters.)

Exercise 4 (S)

In this exercise (as well as later on this sheet), we represent $m \times n$ matrices with two-dimensional Java arrays of type `double[][]`. You may want to revisit the code from Lesson 15 for examples. Some template code is given in the class `MatrixOps`.

(a) Write a function `constantMatrix` that takes no parameters and returns a representation of the 3×4 matrix

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 3 & 4 & 5 & 0.6 \end{pmatrix}.$$

(b) Write a function `matrixSum` that takes two matrices, A and B, as its input, and returns their sum $A+B$ as a `double[][]` array.

(c) Write a function `matrixDifference` that takes two matrices, A and B, as its input, and returns their difference A-B as a `double[][]` array.

Throughout this exercise, you may assume that all array input parameters are non-null, contain valid $m \times n$ matrices, and that the number of rows and columns of the input matrices are consistent with the computation.

Exercise 5 (I)

In three-dimensional space, rotations around the x, y and z axis by an angle θ are described by the following rotation matrices R_x, R_y and R_z, respectively:

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{pmatrix}, \tag{6}$$

$$R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}, \tag{7}$$

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{8}$$

(a) Write a function `rotationMatrix` that takes an integer (the number of the axis) and a `double` number (θ) as its input, and returns the corresponding rotation matrix as a 3×3 `double[][]` array. Regarding the axis, the numbers $1, 2, 3$ should correspond to the x, y, z axis respectively; you can ignore any other values.

(b) Write a function `rotateVector` that rotates a vector v around the x axis by an angle θ_x, then around the y axis by an angle θ_y, then around the z axis by an angle θ_z. That is, the function computes $R_z(\theta_z)R_y(\theta_y)R_x(\theta_x)v$. The parameter types are `double[]` for the vector v, and three `double` parameters for θ_x, θ_y, θ_z in this order. The output type is `double[]`. You do not need to make any consistency checks on the input parameters (i.e., you can assume that the input vector is a valid 3-vector).

Hint: $R_z(\theta_z)R_y(\theta_y)R_x(\theta_x)v = R_z(\theta_z)\big(R_y(\theta_y)\big(R_x(\theta_x)v\big)\big)$, that is, multiplying a matrix with a vector is sufficient here; you don't need to multiply matrices.

(c) Write a function `rotateThisVector`, without parameters, that takes the vector $v = (1, 3, 2)$ and rotates it by $\pi/3$ around the x axis, then by $-\pi/6$ around the z axis. The resulting vector should be returned as a `double[]`.

The function `matrixTimesVector` from Lesson 15 is already contained in the code template and can be used. You might find it easier to do first (b) and then (c), or first (c) then (b); both approaches are valid.

Exercise 6 (I)

This exercise develops further functions that operate on matrices. You might want to look at `matrixTimesVector` from the previous exercise as an example. As before, we store $m \times n$ matrices in a `double[][]` array, where the first index indicates the row and the second index indicates the column of an entry. Please use the class `MoreMatrixOps` for your code.

a) The transpose of an $m \times n$ matrix A is an $n \times m$ matrix A^T defined by $(A^T_{ij}) = A_{ji}$, $0 \le i < m$, $0 \le j < n$. Write a function `transpose` that computes the transpose of a matrix. Both the (only) parameter and the return value are `double[][]` arrays.

b) The product of an $m \times n$ matrix A and an $n \times p$ matrix B is given by

$$(A \cdot B)_{ik} = \sum_{0 \le j < n} A_{ij} B_{jk}. \tag{9}$$

Write a function `matrixProduct` that computes the product of two matrices, given as two `double[][]` parameters, and returns the result as a `double[][]`.

If any of the input parameters is `null`, the corresponding function should return `null`. If the matrix dimensions in the function `matrixProduct` do not match, the result should be `null` as well.

Exercise 7 (A)

Just as two vectors in 3-dimensional space span a 2-dimensional plane through the origin, k vectors in n-dimensional space span a k-dimensional hyperplane through the origin. In describing such a hyperplane, one often prefers to use *orthonormal* vectors, that is, vectors of norm 1 which are mutually orthogonal.

Finding these orthonormal vectors can be achieved by the so-called *Gram-Schmidt process*. This construction takes k vectors $u^{(0)}, \ldots, u^{(k-1)} \in \mathbb{R}^n$ as its input, and produces orthonormal vectors $v^{(0)}, \ldots, v^{(k-1)} \in \mathbb{R}^n$ which span the same hyperplane as the u's. If the u's were already orthonormal, then $u^{(j)} = v^{(j)}$ for all j.

Read about the Gram-Schmidt process, for example on Wikipedia. Then implement the Gram-Schmidt process in a function `orthonormalize`, which takes a parameter of type `double[][]` and produces a result of the same type. The input parameter (say, `u`) should be taken as an array of vectors, i.e., `u[2][3]` would be the 3-component of the vector $u^{(2)}$.

You do not need to perform consistency checking on the input parameters, i.e., you can assume that all input vectors $u^{(j)}$ have the same dimension n, that $0 < k \le n$, and that the input vectors actually span a hyperplane of dimension k (none of them happen to be collinear, etc.).

The functions for computing with vectors as developed in Exercise 3 might come in handy.